

Technology Stack

ParSU e-Feedback — Integrated Student Complaint and Feedback Management System with Institutional Analytics

This document lists every technology the system runs on, from the framework it is built with down to the smallest supporting service, with the version in use and what each one is actually responsible for. Where a dependency is installed but not yet imported by application code, that is stated plainly rather than implied.

The stack at a glance

Language & runtime	TypeScript 5.9 (strict) on Node.js, React 19.2
Framework	Next.js 16.2 App Router — Server Components, Route Handlers, Server Actions
Interface	TailwindCSS 4.3 with shadcn/ui conventions, Radix UI, Lucide icons, Recharts
Database	MongoDB Atlas via Mongoose 9.7 (MongoDB driver 7.4)
Authentication	Auth.js (next-auth 5 beta) with JWT sessions and bcrypt password hashing
File storage	Cloudflare R2, addressed through the AWS S3 SDK with presigned URLs
Email	Resend, with templates authored in React Email
Redis & scheduling	Upstash Redis for rate limiting; Vercel Cron for the daily SLA check
Exports	ExcelJS for spreadsheets, PDFKit for PDFs, hand-built CSV
Testing	Vitest with mongodb-memory-server for units and integration, Playwright for end-to-end
Tooling	ESLint 9, Prettier 3, npm, Git

Generated 2026-09-03. Versions are the ones actually installed in node_modules, not the ranges declared in package.json. A complete package inventory appears in the appendix.

1. Language, Runtime and Core Libraries

The foundation everything else is written against.

- **TypeScript** 5.9.3
The entire codebase is TypeScript in strict mode — no plain JavaScript source files. Types are shared between server and client, and Mongoose schemas infer their document types rather than declaring them twice. 'npm run typecheck' runs the compiler with no emit as a standalone gate.
Used in: tsconfig.json, every .ts and .tsx file
- **React** 19.2.4
The user-interface library. The project uses the modern split: most screens are React Server Components that fetch on the server, with 'use client' reserved for genuinely interactive pieces such as forms, dropdowns and charts.
*Used in: src/app/**, src/components/***
- **React DOM** 19.2.4
The browser renderer for React, plus the hooks used by form components (useFormStatus, useTransition).
Used in: client components
- **Node.js** 24.x (local)
The server runtime for route handlers, Server Actions and the build. Node-only libraries such as Mongoose and bcrypt are deliberately kept out of the Edge-runtime code path.
Used in: runtime
- **npm** bundled with Node
Package manager. package-lock.json pins the exact dependency tree so installs are reproducible.
Used in: package.json, package-lock.json

2. Application Framework

Next.js supplies the routing, rendering, bundling and server endpoints — it is the single framework the whole system is built on.

- **Next.js** 16.2.10
The App Router framework. It provides: file-based routing (folders as URL segments), Server Components for data fetching, Route Handlers as the HTTP API, Server Actions for form submissions, layout and loading conventions for per-role shells and skeleton states, streaming, and the production build.
*Used in: src/app/**, next.config.ts*
- **Next.js request interceptor (proxy)** built in
Runs before any route resolves. It reads the session token and redirects unauthenticated visitors to login and wrong-role users to their own home — the first, coarse layer of access control, backed by a pure route/role table.
Used in: src/proxy.ts, src/middleware/rbac.ts
- **next/font (Google Fonts)** built in
Self-hosts the two typefaces at build time so there is no external font request at runtime: Plus Jakarta Sans for the interface and IBM Plex Mono for ticket numbers and identifiers.
Used in: src/app/layout.tsx
- **Vercel Cron** platform feature
The scheduler. vercel.json registers a daily job that calls the SLA check endpoint, which then finds approaching and breached deadlines, escalates and notifies. The endpoint also accepts an external trigger carrying the shared CRON_SECRET.
Used in: vercel.json, src/app/api/cron/sla-check/route.ts

3. Interface and Styling

The visual layer: a utility-first stylesheet, a small set of accessible primitives, and a fixed light theme expressed entirely in CSS variables.

- **TailwindCSS** 4.3.2
The styling system. Version 4 is configured from CSS rather than a JavaScript config file — globals.css imports Tailwind and then declares the theme tokens directly.
Used in: src/app/globals.css and every component
- **@tailwindcss/postcss** 4.3.2
The PostCSS plugin that compiles Tailwind during the Next.js build.
Used in: postcss.config.mjs

- **shadcn/ui (conventions)** `components.json`
Not a runtime dependency but a code-generation convention the project follows: 'new-york' style, neutral base, CSS-variable theming and the `@/components`, `@/lib` and `@/hooks` aliases. The theme in `globals.css` uses shadcn's exact variable names, so any generated component matches the palette with no extra theming.
Used in: `components.json`, `src/components/ui/`
- **Radix UI — Select** 2.3.7
The accessible, unstyled primitive behind the app's dropdowns, wrapped once as a themed component and reused everywhere a select appears.
Used in: `src/components/ui/select.tsx`
- **Lucide React** 1.23.0
The icon set, used across roughly 79 files — navigation, status markers, dashboard tiles, buttons and empty states.
Used in: `src/components/**`, `src/app/**`
- **clsx** 2.1.1
Builds conditional class strings.
Used in: `src/lib/utils.ts`
- **tailwind-merge** 3.6.0
Resolves conflicting Tailwind classes so a component's default can be overridden by a caller. Paired with `clsx` in the `cn()` helper that nearly every component uses.
Used in: `src/lib/utils.ts`
- **class-variance-authority** 0.7.1
Installed as part of the shadcn/ui toolchain for typed component variants. Present in the dependency tree but not currently imported by application code.
Used in: `dependency only`

4. Data Visualisation

Charts for the dean, QA and admin analytics screens.

- **Recharts** 3.9.2
The charting library. It renders the monthly complaint-volume trend line, the complaints-per-category bar chart, the priority distribution pie, and the QA dashboard's donut, bar-list and trend charts. All chart components are client components fed by aggregated server data.
Used in: `src/components/analytics/**`, `src/components/qa/charts/**`
- **react-is** 19.2.8
A React internals helper that Recharts requires. Pinned explicitly so the chart library resolves a version compatible with React 19.
Used in: `transitive requirement`
- **Custom SVG heat map** `in-house`
The per-office SLA compliance grid is hand-built rather than charted, since it is a table of coloured cells rather than a plotted series.
Used in: `src/components/qa/SlHeatmap.tsx`

5. Database and Data Access

A document database, accessed through one connection and one modelling layer.

- **MongoDB Atlas** cloud service
The hosted database. Every collection in the system — users, offices, categories, complaints, timeline, notes, assignments, attachments, feedback, notifications, audit logs, routing rules, SLA rules, generated reports, reset tokens, counters and settings — lives here.
Used in: MONGODB_URI
- **Mongoose** 9.7.3
The object-document mapper. It defines every collection's schema, enums, indexes and references, infers the TypeScript types from those schemas, and runs the aggregation pipelines behind the dashboards and reports.
*Used in: src/models/**, src/features/***
- **MongoDB Node driver** 7.4.0
The underlying driver Mongoose speaks to the cluster with. Pinned directly so the driver version is explicit rather than whatever a transitive resolution picks.
Used in: dependency of mongoose
- **Connection singleton** in-house
Mongoose connections are cached on the global object, so serverless invocations and development hot-reloads reuse one connection instead of exhausting the Atlas connection limit.
Used in: src/lib/db.ts
- **Reference-integrity plugin** in-house
A Mongoose plugin that validates every ObjectId reference actually points at an existing document — a check the library does not perform on its own.
Used in: src/lib/mongoose-ref-integrity.ts
- **Atomic counters** in-house
Ticket numbers are issued by an upsert with \$inc against a counter document, which is atomic in MongoDB and therefore safe when two students submit at the same moment.
Used in: src/models/Counter.ts

6. Authentication, Authorisation and Security

Five roles, session-based access, and layered checks that never rely on a single gate.

- **Auth.js (next-auth)** 5.0.0-beta.31
The authentication framework. It runs the Credentials provider that verifies email and password against the database, issues and reads the session, and drives sign-in, sign-out and the session endpoints. Its configuration is deliberately split into an Edge-safe half and a Node-only half.
Used in: src/lib/auth.config.ts, src/lib/auth.ts
- **JWT sessions** via Auth.js
Sessions are stateless signed tokens rather than database rows, with an inactivity expiry driven by configuration. A token-version field on the user lets an administrator invalidate live sessions immediately.
Used in: src/features/auth/services/session.service.ts
- **jose** 6.2.3
The JSON Web Token signing and verification library Auth.js uses underneath. It is not called directly by application code.
Used in: transitive requirement
- **bcryptjs** 3.0.3
Password hashing at ten salt rounds. Every hash and verification goes through one helper, so no call site can accidentally handle a password in plaintext.
Used in: src/lib/password.ts, src/features/auth/services/password.service.ts
- **Role-based access control** in-house
Three layers: a route/role table applied by the request interceptor, reusable session and role guards at the top of every route handler, and scope checks inside each handler for office- or college-level ownership that a URL cannot express.
Used in: src/middleware/rbac.ts, src/lib/api-guards.ts
- **Account logout** in-house
A failed-attempt counter with a temporary logout window, both thresholds configurable at runtime.
Used in: src/features/auth/services/auth.service.ts

- **Single-use reset tokens** in-house
Password-reset tokens are random, stored hashed, expiring and consumed on use.
Used in: src/models/PasswordResetToken.ts
- **Audit trail** in-house
An insert-only log of every sensitive action, recording actor, action, target, before and after values, IP address and timestamp.
Used in: src/models/AuditLog.ts
- **Environment validation** in-house
A single Zod schema parses every environment variable at import time, so a missing secret fails immediately with a readable message instead of surfacing as an undefined value deep in a request.
Used in: src/lib/env.ts

7. Validation

One validation library, used for both configuration and every piece of user input.

- **Zod** 4.4.3
Schema validation and type inference, used in roughly 17 modules. It validates registration, login and password-reset input, complaint creation and updates, assignment payloads, attachment presign requests, feedback, broadcasts, report parameters and every administrator form, as well as the environment itself. Because the inferred types come from the same schema, the validated shape and the TypeScript type can never drift apart.
*Used in: src/features/**/schemas/**, src/lib/env.ts*

8. File Storage and Attachments

Uploads bypass the application server entirely and go straight to object storage.

- **Cloudflare R2** cloud service
S3-compatible object storage holding complaint attachments and generated report files. Chosen for having no egress charges.
*Used in: R2_ * environment variables*
- **@aws-sdk/client-s3** 3.1079.0
The S3 client used to talk to R2, since R2 implements the S3 API.
Used in: src/lib/r2.ts
- **@aws-sdk/s3-request-presigner** 3.1079.0
Generates short-lived presigned URLs. The browser uploads directly to R2 with one, and downloads are served through another, so files never pass through the application server and links cannot be shared indefinitely.
Used in: src/app/api/uploads/presign/route.ts
- **Upload policy** in-house
File type and size are checked against a configurable MIME allow-list and size ceiling before any URL is issued; only metadata is stored in MongoDB.
*Used in: src/features/attachments/**, src/models/Attachment.ts*

9. Email and Notifications

Transactional mail written as React components, delivered by a single provider.

- **Resend** 6.17.1
The email delivery service and its SDK. It sends every transactional message the system produces.
Used in: src/lib/resend.ts, src/features/notifications/services/email.service.ts
- **React Email** 6.6.6
Lets the templates be written as React components with a shared layout and rendered to email-safe HTML at send time, instead of maintaining raw HTML strings.
*Used in: emails/***
- **Template set** seven templates
Complaint submitted, assigned, updated and resolved; SLA warning; escalation; and password reset.
Used in: emails/.tsx*
- **In-app notifications** in-house
Stored notification records with type, recipient, link and read state, surfaced through the bell and the inbox page, plus an administrator broadcast to a whole role.
*Used in: src/features/notifications/***

10. Redis, Rate Limiting and Background Work

The supporting services around the request path.

- **Upstash Redis** 1.38.0
Serverless Redis over HTTP, reachable from any runtime without a persistent socket. It backs the rate limiter.
Used in: src/lib/rate-limit.ts
- **@upstash/ratelimit** 2.0.8
The rate-limiting algorithm on top of that Redis instance, applied to login, registration and other abusable endpoints.
Used in: src/lib/rate-limit.ts
- **@upstash/qstash** 2.11.1
A message and scheduling service. It is installed and the SLA endpoint accepts its trigger headers as an alternative to Vercel Cron, but no application code imports it at present.
Used in: dependency only
- **@upstash/workflow** 1.3.2
Durable multi-step workflow runner from the same provider, installed alongside QStash for the same scheduling role. Not currently imported.
Used in: dependency only
- **Shared-secret cron guard** in-house
The SLA endpoint accepts either the platform's cron signature or a request carrying CRON_SECRET, and guards against overlapping runs.
Used in: src/app/api/cron/sla-check/route.ts

11. Reporting and Exports

The same dataset rendered into three downloadable formats.

- **ExcelJS** 4.4.0
Writes real .xlsx workbooks with typed columns and headers for the Excel export.
Used in: src/features/reports/services/report-file.service.ts
- **PDFKit** 0.19.1
Renders the PDF export. Because it loads its font-metric files from disk at runtime, next.config.ts marks it as a server-external package so the bundler leaves those paths intact.
Used in: src/features/reports/services/report-file.service.ts, next.config.ts
- **@types/pdfkit** 0.17.6
TypeScript definitions for PDFKit, which ships none of its own.
Used in: development
- **CSV writer** in-house
The comma-separated export is generated directly, needing no library.
- **Report pipeline** in-house
An aggregation builds the filtered dataset, the file service renders it, the upload service stores it in R2, and a database record tracks the creator, filters, format, status and download location.
*Used in: src/features/reports/services/***

12. Testing

Two suites: fast tests around the domain logic, and browser tests that exercise the real application.

- **Vitest** 4.1.10
The unit and integration test runner. It covers the lifecycle state machine, routing, SLA calculation, ticket numbering, password hashing, the role table, report generation and the schemas.
*Used in: vitest.config.ts, src/**/*.test.ts*
- **mongodb-memory-server** 11.2.0
Boots a real MongoDB instance in memory for integration tests, so model behaviour, indexes and referential integrity are proven against the actual database engine rather than a mock.
Used in: src/test/setup-db.ts
- **@vitejs/plugin-react** 6.0.3
Lets Vitest compile JSX and React modules.
Used in: vitest.config.ts
- **Playwright** 1.61.1
The end-to-end runner. It seeds the database once, logs in as each of the five roles and saves those sessions, then drives Chromium through authentication, submission, the status lifecycle, edit and withdrawal, scoping, attachments, notes, notifications, ratings, analytics, reports and administrator management.
*Used in: playwright.config.ts, tests/e2e/***
- **Seed endpoint** in-house
A development-only route that resets the database to a known state — fixed accounts, offices, categories, roughly 300 complaints and 24 feedback rows — so every end-to-end run starts identically.
Used in: src/app/api/dev/seed/route.ts
- **@testing-library/react** 16.3.2
Component-testing library, installed for React component tests. Present in the toolchain but not currently imported by any test.
- **@testing-library/jest-dom** 6.9.1
The matching set of DOM assertions for that library. Installed, not currently imported.
- **supertest** 7.2.2
HTTP assertion library for testing endpoints directly. Installed, not currently imported — API behaviour is covered through the integration and Playwright suites instead.

13. Code Quality and Developer Tooling

What keeps the codebase consistent between contributors and runs.

- **ESLint** 9.39.4
Linting, using the flat config format.
Used in: eslint.config.mjs
- **eslint-config-next** 16.2.10
The Next.js rule set, including its React and accessibility rules, matched to the framework version.
Used in: eslint.config.mjs
- **Prettier** 3.9.4
Formatting, run over the whole project by 'npm run format' and verifiable in CI by 'format:check'.
Used in: .prettierrc.json
- **prettier-plugin-tailwindcss** 0.8.0
Sorts Tailwind class names into a canonical order so class lists stay readable and diffs stay small.
Used in: .prettierrc.json
- **@types/node, @types/react, @types/react-dom** 20.19 / 19.2 / 19.2
TypeScript definitions for the runtime and React.
Used in: development
- **Path alias** @/* -> src/*
One import alias across the project, honoured by TypeScript, Next.js, Vitest and the shadcn configuration alike.
Used in: tsconfig.json, vitest.config.ts
- **Git** version control
The repository's history; .gitignore keeps secrets, build output, dependencies and Playwright artefacts out of it.

- **Visual Studio Code** editor

Shared workspace settings commit the formatter and format-on-save behaviour so every contributor writes identically formatted code.

Used in: .vscode/settings.json

14. Hosting and External Services

Everything the deployed system depends on outside its own code, and the configuration each one needs.

- **Vercel** hosting platform

Runs the Next.js application and its serverless functions, holds the environment variables per environment, and provides the cron scheduler that drives the daily SLA check.

Used in: vercel.json

- **MongoDB Atlas** database

The managed cluster. Needs MONGODB_URI; separate clusters are used for development and production.

- **Cloudflare R2** object storage

Attachments and generated reports. Needs the account ID, an access key pair, the bucket name and the public base URL.

- **Resend** email delivery

Transactional mail from a verified sender domain. Needs the API key and the from address.

- **Upstash Redis** serverless Redis

Rate limiting. Needs the REST URL and token.

- **Auth.js secret** configuration

AUTH_SECRET signs the session tokens; AUTH_URL names the canonical application URL.

- **CRON_SECRET** configuration

The shared secret the scheduled SLA endpoint checks on every hit.

- **Runtime thresholds** configuration

Lockout attempts and duration, session length, upload size and MIME allow-list and the ticket prefix begin as environment values and are now editable at runtime by an administrator through the settings singleton, with no redeploy needed.

Used in: src/models/Settings.ts

Appendix — Complete Package Inventory

Every package declared in package.json, with the version installed in node_modules. 'Runtime' packages ship with the application; 'dev' packages are used only to build, lint or test it.

Package	Version	Type	Purpose
@aws-sdk/client-s3	3.1079.0	runtime	S3 client for Cloudflare R2 storage
@aws-sdk/s3-request-presigner	3.1079.0	runtime	Presigned upload and download URLs
@radix-ui/react-select	2.3.7	runtime	Accessible dropdown primitive
@upstash/qstash	2.11.1	runtime	Message and schedule service (installed, unused)
@upstash/ratelimit	2.0.8	runtime	Rate limiting on abusable endpoints
@upstash/redis	1.38.0	runtime	Serverless Redis over HTTP
@upstash/workflow	1.3.2	runtime	Durable workflow runner (installed, unused)
bcryptjs	3.0.3	runtime	Password hashing and verification
class-variance-authority	0.7.1	runtime	Typed component variants (installed, unused)
clsx	2.1.1	runtime	Conditional class-name builder
exceljs	4.4.0	runtime	Excel (.xlsx) report export
jose	6.2.3	runtime	JWT signing used by Auth.js
lucide-react	1.23.0	runtime	Icon set used across the interface
mongoose	7.4.0	runtime	MongoDB Node driver under Mongoose
mongoose	9.7.3	runtime	Schemas, models and aggregation
next	16.2.10	runtime	Application framework (App Router)
next-auth	5.0.0-beta.31	runtime	Auth.js authentication and sessions
pdfkit	0.19.1	runtime	PDF report export
react	19.2.4	runtime	User-interface library
react-dom	19.2.4	runtime	Browser renderer for React
react-email	6.6.6	runtime	Email templates as React components
react-is	19.2.8	runtime	React internals helper required by Recharts
recharts	3.9.2	runtime	Charts on the analytics dashboards
resend	6.17.1	runtime	Transactional email delivery
tailwind-merge	3.6.0	runtime	Resolves conflicting Tailwind classes
zod	4.4.3	runtime	Schema validation for input and environment
@playwright/test	1.61.1	dev	End-to-end browser test runner
@tailwindcss/postcss	4.3.2	dev	Tailwind compilation in the build
@testing-library/jest-dom	6.9.1	dev	DOM assertions (installed, unused)
@testing-library/react	16.3.2	dev	Component testing (installed, unused)
@types/node	20.19.43	dev	Node.js type definitions
@types/pdfkit	0.17.6	dev	PDFKit type definitions
@types/react	19.2.17	dev	React type definitions
@types/react-dom	19.2.3	dev	React DOM type definitions
@vitejs/plugin-react	6.0.3	dev	JSX support inside Vitest
eslint	9.39.4	dev	Linting
eslint-config-next	16.2.10	dev	Next.js lint rule set
mongodb-memory-server	11.2.0	dev	In-memory MongoDB for integration tests
prettier	3.9.4	dev	Code formatting
prettier-plugin-tailwindcss	0.8.0	dev	Sorts Tailwind class names
supertest	7.2.2	dev	HTTP endpoint testing (installed, unused)
tailwindcss	4.3.2	dev	Utility-first styling system
typescript	5.9.3	dev	Language and compiler

Package	Version	Type	Purpose
vitest	4.1.10	dev	Unit and integration test runner

Totals: 26 runtime packages and 18 development packages, plus five external services (MongoDB Atlas, Cloudflare R2, Resend, Upstash Redis and the Vercel hosting platform).